

Лекция №2

Условные конструкции в JavaScript

Цель лекции

Научиться выполнять различные действия в программе в зависимости от заданных условий.

- сравнивать значения;
- использовать логические операторы;
- создавать условия с помощью if, else if, else;
- объединять несколько условий;
- использовать switch;
- понимать назначение тернарного оператора.

1. Условия в программах

Программа часто должна принимать решение: разрешить покупку, проверить наличие товара, определить скидку или права пользователя. Для этого используются условные конструкции.

2. Операторы сравнения

Результатом сравнения является логическое значение true или false. Основные операторы: > — больше; < — меньше; >= — больше или равно; <= — меньше или равно; === — строго равно; !== — строго не равно.

```
const price = 5000;
console.log(price > 3000);
console.log(price === 5000);
```

3. Строгое сравнение ===

Оператор == может автоматически преобразовывать типы. Оператор === сравнивает и значение, и тип. В курсе рекомендуется использовать === и !==.

```
console.log(5 == "5"); // true
console.log(5 === "5"); // false
```

4. Логические операторы

&& (И) требует истинности обоих условий; || (ИЛИ) — хотя бы одного; ! (НЕ) меняет логическое значение на противоположное.

```
const price = 5000;
const stock = 10;
console.log(price > 1000 && stock > 0);
```

5. Условная конструкция if

Код внутри фигурных скобок выполняется только тогда, когда условие истинно.

```
const stock = 5;
if (stock > 0) {
  console.log("Товар есть в наличии");
}
```

6. Конструкция if...else

Если условие истинно, выполняется блок if, иначе — блок else.

```
const stock = 0;
if (stock > 0) {
  console.log("Товар есть в наличии");
} else {
  console.log("Товар отсутствует");
}
```

7. Несколько условий: else if

Если вариантов несколько, используются дополнительные проверки else if. Условия проверяются сверху вниз, поэтому их порядок имеет значение.

```
const total = 8000;
if (total >= 10000) {
  console.log("Скидка 15%");
} else if (total >= 5000) {
  console.log("Скидка 10%");
} else {
  console.log("Скидки нет");
}
```

8. Составные условия

Несколько проверок можно объединить логическими операторами.

```
const stock = 10;
const count = 3;
if (stock > 0 && count <= stock) {
  console.log("Заказ можно оформить");
} else {
  console.log("Недостаточно товара");
}
```

9. Проверка пользователя

В backend-приложениях условия применяются для проверки ролей и состояния пользователя.

```
const role = "admin";
const isBlocked = false;
if (role === "admin" && !isBlocked) {
  console.log("Доступ разрешён");
} else {
  console.log("Доступ запрещён");
}
```

10. Вложенные условия

Внутри одного if можно разместить другой if. Это допустимо, но большое количество вложений усложняет чтение программы.

```
if (stock > 0) {
  if (count <= stock) {
    console.log("Заказ оформлен");
  }
}
```

11. Конструкция switch

switch удобно использовать, когда одно значение сравнивается с несколькими фиксированными вариантами. break завершает выбранный case, default выполняется, если подходящего варианта нет.

```
const status = "paid";
switch (status) {
  case "new":
    console.log("Новый заказ");
    break;
  case "paid":
    console.log("Заказ оплачен");
    break;
  default:
    console.log("Неизвестный статус");
}
```

12. Тернарный оператор

Для короткого простого условия можно использовать запись: условие ? значение1 : значение2. Для начинающих сложные условия лучше записывать через if...else.

```
const stock = 5;
const message = stock > 0 ? "В наличии" : "Нет в наличии";
console.log(message);
```

13. Комплексный пример

Пример простой логики интернет-магазина:

```
const productName = "Ноутбук";
const price = 65000;
const stock = 5;
const count = 2;

if (stock === 0) {
  console.log(`Товар "${productName}" отсутствует`);
} else if (count > stock) {
  console.log("Недостаточно товара на складе");
} else {
  const total = price * count;
  console.log(`Сумма заказа: ${total} руб.`);
}
```

14. Типичные ошибки

Не следует использовать = вместо === в условии; ставить точку с запятой сразу после if; располагать более общее условие раньше более строгого.

Краткие выводы

Операторы сравнения: >, <, >=, <=, ===, !==. Логические операторы: &&, ||, !. Основная конструкция ветвления — if...else; для нескольких проверок используется else if. switch удобен для фиксированных вариантов, а тернарный оператор — для коротких условий. В backend-разработке условия применяются для проверки данных, прав пользователя, наличия товара и статусов операций.